

SWE 307

Big Data

Week – 2

Project 1 Help

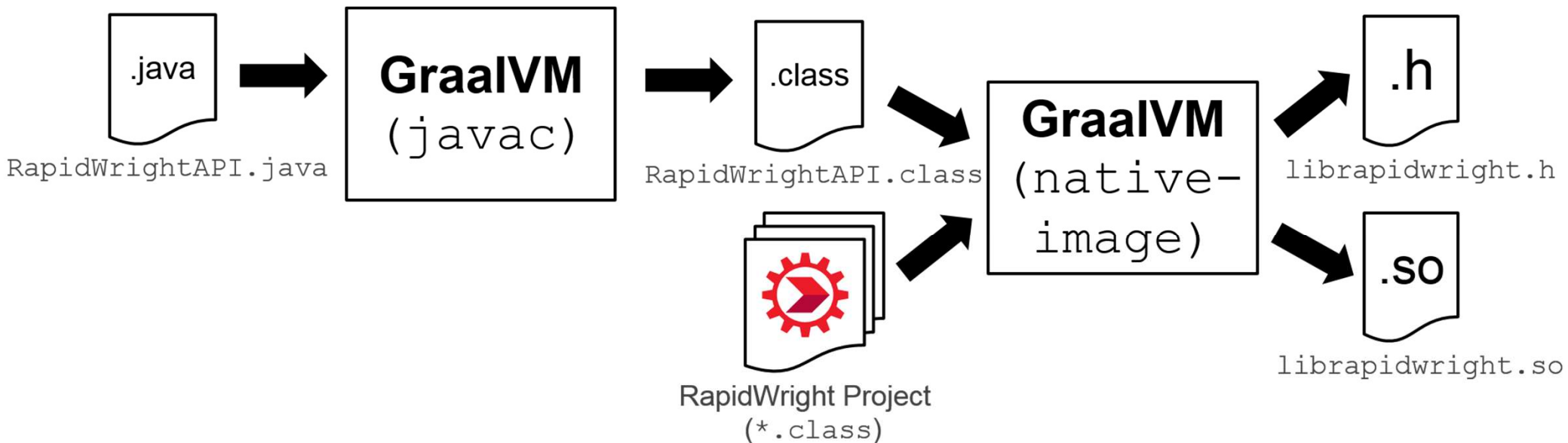
Ahmet Özmen
2026

Overview

- GraalVM, Polyglot
- R and Python on top of GraalVM
- Project structure, what is expected?
- Example project with GraalVM and R

GraalVM

Build faster, smaller, leaner applications



Simple Application Example

Using **GraalVM Native Image** you can compile Java bytecode into a platform-specific, self-contained, native executable to achieve faster startup and a smaller footprint for your application.

Compile this *HelloWorld.java* application to bytecode and then build a native executable:

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

Copy

```
javac HelloWorld.java
```

Copy

```
native-image HelloWorld
```

Copy

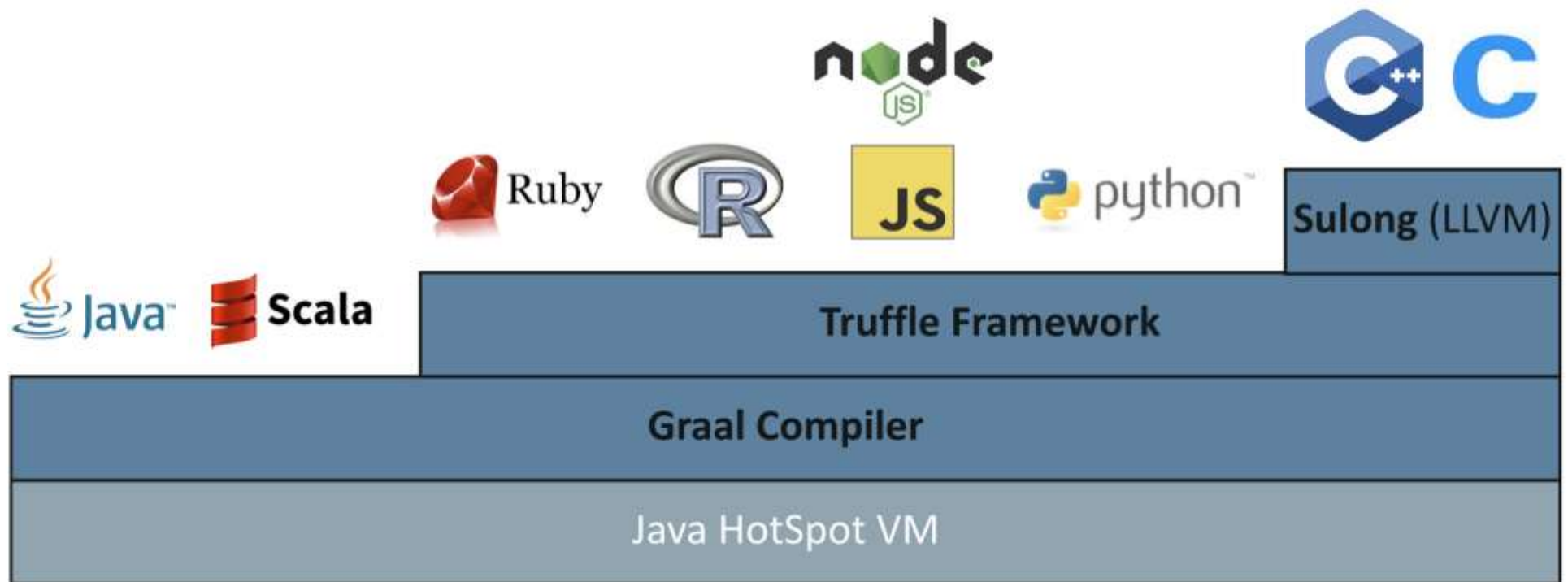
The last command generates an executable file named *helloworld* in the current working directory. Invoking it runs the natively-compiled code of the **HelloWorld** class as follows:

```
./helloworld  
Hello, World!
```

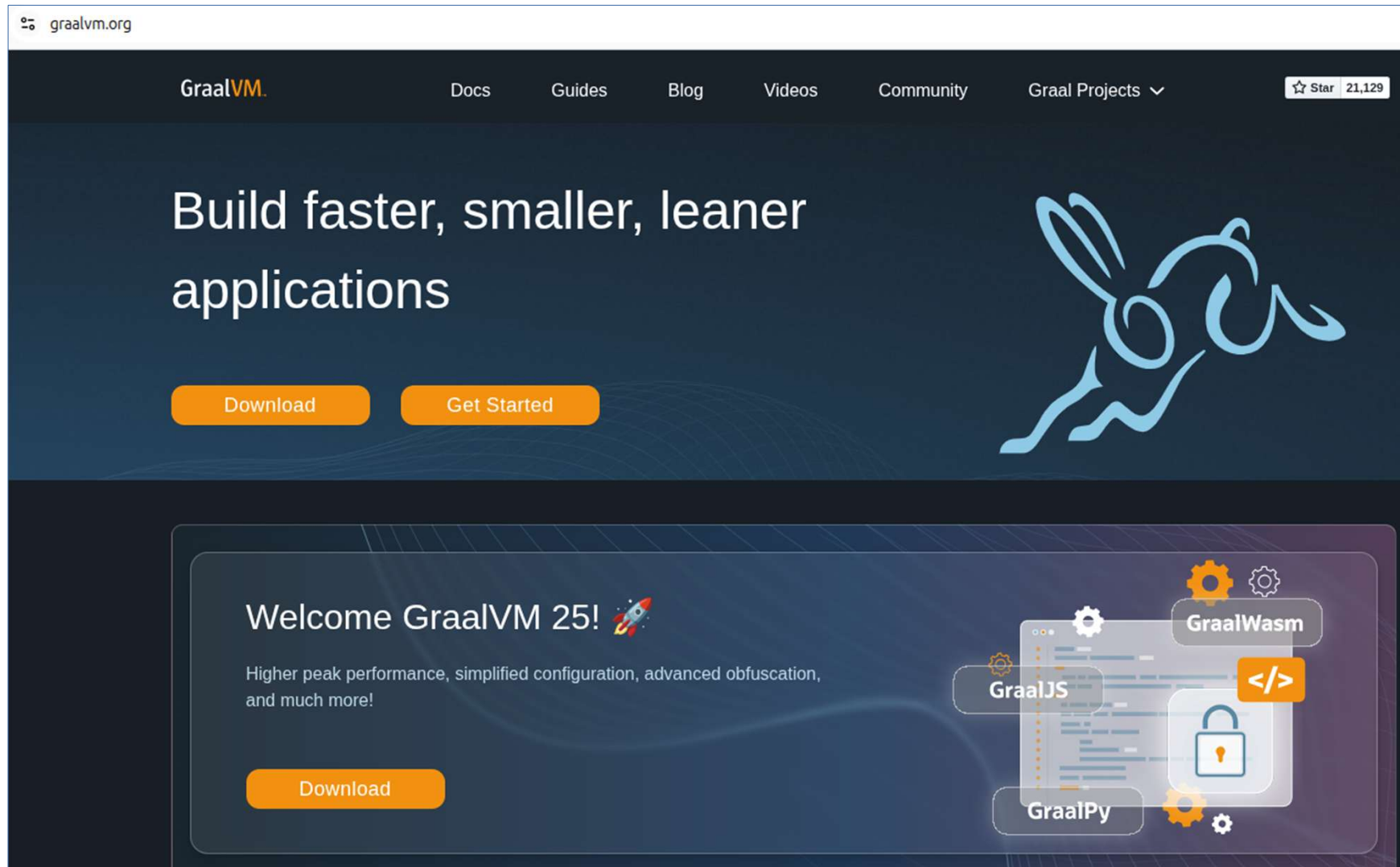
Copy

<https://www.graalvm.org/latest/getting-started/>

POLYGLOT: Java and R or Python



Latest GraalVM



Project 1: We are going to use GraalVM version 22.3.0

<https://www.graalvm.org/>

Install GraalVM 22.3.0 and R

1) Download GraalVM into a special folder:

```
ozmen@asus:~/GraalVM$ bash <(curl -sL https://get.graalvm.org/jdk) graalvm-ce-java17-22.3.0
```

2) Modify your .bashrc file properly. Then source it.

```
export PATH="/home/ozmen/GraalVM/graalvm-ce-java17-22.3.0/bin:$PATH"
export JAVA_HOME="/home/ozmen/GraalVM/graalvm-ce-java17-22.3.0"
```

3) \$ source .bashrc

4) Run GraalVM updater: gu list and gu available

```
ozmen@asus:~$ gu list
ComponentId          Version      Component name      Stability      Origin
-----
graalvm              22.3.0      GraalVM Core        Supported
native-image         22.3.0      Native Image        Early adopter  github.com
ozmen@asus:~$ gu available
Downloading: Component catalog from www.graalvm.org
ComponentId          Version      Component name      Stability      Origin
-----
espresso             22.3.0      Java on Truffle     Supported      github.com
espresso-llvm        22.3.0      Java on Truffle LLVM Java librSupported      github.com
js                   22.3.0      Graal.js            Supported      github.com
llvm                 22.3.0      LLVM Runtime Core   Supported      github.com
llvm-toolchain       22.3.0      LLVM.org toolchain  Supported      github.com
native-image         22.3.0      Native Image        Early adopter  github.com
native-image-llvm-backend22.3.0      Native Image LLVM Backend Early adopter (experimental) github.com
nodejs               22.3.0      Graal.nodejs        Supported      github.com
python               22.3.0      GraalVM Python      Experimental   github.com
R                    22.3.0      FastR               Experimental   github.com
ruby                 22.3.0      TruffleRuby          Experimental   github.com
visualvm             22.3.0      VisualVM            Experimental   github.com
wasm                  22.3.0      GraalWasm           Experimental   github.com
ozmen@asus:~$
```

Install R on GraalVM

5) Run GraalVM updater: gu install R

```
ozmen@asus:~$ gu install R
Downloading: Component catalog from www.graalvm.org
Processing Component: FastR
Processing Component: LLVM.org toolchain
Processing Component: LLVM Runtime Core
Additional Components are required:
  LLVM.org toolchain (org.graalvm.llvm-toolchain, version 22.3.0), required by: FastR (org.graalvm.R)
  LLVM Runtime Core (org.graalvm.llvm, version 22.3.0), required by: FastR (org.graalvm.R)
Downloading: Component R: FastR from github.com
[#####]█
```

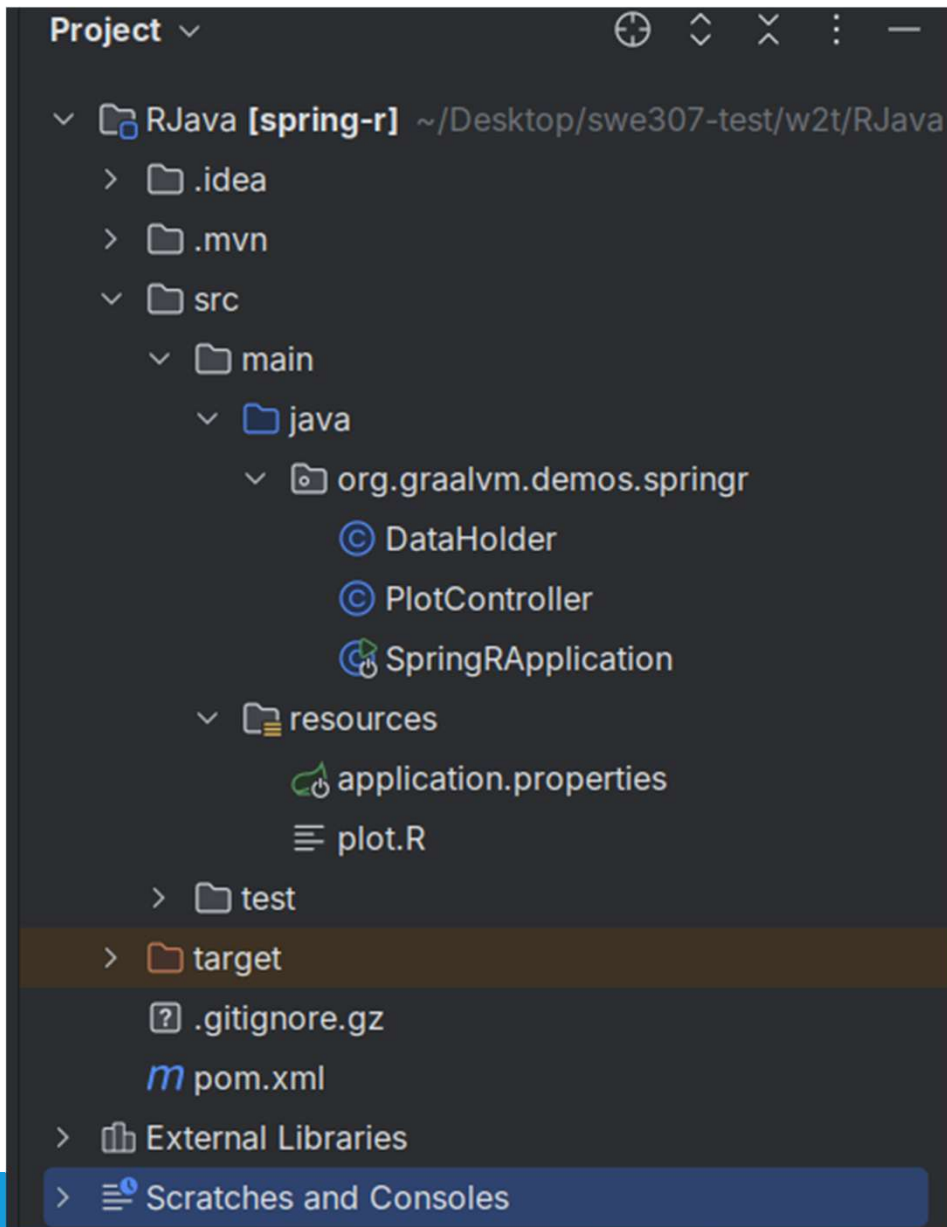
NOTE: GraalVM updater (gu) is not available after GraalVM 22.3.0!

```
ozmen@asus:~$ gu list
```

ComponentId	Version	Component name	Stability
-----	-----	-----	-----
graalvm	22.3.0	GraalVM Core	Supported
R	22.3.0	FastR	Experimental
llvm	22.3.0	LLVM Runtime Core	Supported
llvm-toolchain	22.3.0	LLVM.org toolchain	Supported
native-image	22.3.0	Native Image	Early adopter

```
ozmen@asus:~$ █
```

Spring Boot with R: Project Structure



This is like model: Data holder class duty is just data carriage

The screenshot shows the source code of the `DataHolder.java` file. The code is as follows:

```
1  > /.../  
41  
42  package org.graalvm.demos.springr;  
43  
44  public class DataHolder { 3 usages  
45      public double value;  
46  
47      public DataHolder(double v) { 1 usage  
48          value = v;  
49      }  
50  }
```

R script

```
plot.R × DataHolder.java SpringRApplication.java PlotController.java
1
2 library(lattice)
3 data <- numeric(100)
4
5 function(dataHolder) {
6     svg()
7     data <- c(data[2:100], dataHolder$value)
8
9     plot <- xyplot(randomData~time,
10         data=data.frame(randomData = data, time = -99:0),
11         main='Random Number Plot',
12         ylab="Random(x)", type = c('l', 'g'), col.line='dark orange')
13     print(plot)
14     svg.off()
15 }
```

Controller

```
19 @Controller
20 public class PlotController {
21     @Value(value = "classpath:plot.R")
22     private Resource rSource;
23     @Autowired
24     private Function<DataHolder, String> plotFunction;
25     @Bean
26     Function<DataHolder, String> getPlotFunction(@Autowired Context ctx)
27         throws IOException {
28         Source source = Source.newBuilder(language: "R", rSource.getURL()).build();
29         return ctx.eval(source).as(Function.class);
30     }
31     public static double x = 0; 3 usages
32     @RequestMapping(value = "/plot", produces = "image/svg+xml")
33     public ResponseEntity<String> load() {
34         HttpHeaders responseHeaders = new HttpHeaders();
35         responseHeaders.set("Refresh", "1");
36
37         String svg = "";
38         synchronized(plotFunction){
39             svg = plotFunction.apply(new DataHolder(v: 10*Math.sin(x)));
40         }
41         x += 0.1;
42         x %= 2 * Math.PI;
43         return new ResponseEntity<String>(svg, responseHeaders, HttpStatus.OK);
44     }
45     @Bean
46     public Context getGaalVMContext() { return Context.newBuilder().allowAllAccess(enabled: true).build(); }
47 }
48 }
```

Build and run the application

Terminal Local × + ▾

```
ozmen@asus:~/Desktop/swe307-test/w2t/RJava$ mvn spring-boot:run
```

```
[INFO] Scanning for projects...
```

```
[INFO]
```

```
[INFO] -----< org.graalvm.demos.spring-r:spring-r >-----
```

```
[INFO] Building spring-r 0.0.1-SNAPSHOT
```

```
[INFO] -----[ jar ]-----
```

```
[INFO]
```

```
[INFO] >>> spring-boot-maven-plugin:2.1.12.RELEASE:run (default-cli) > test-compile @ spring-r >>>
```

```
[INFO]
```

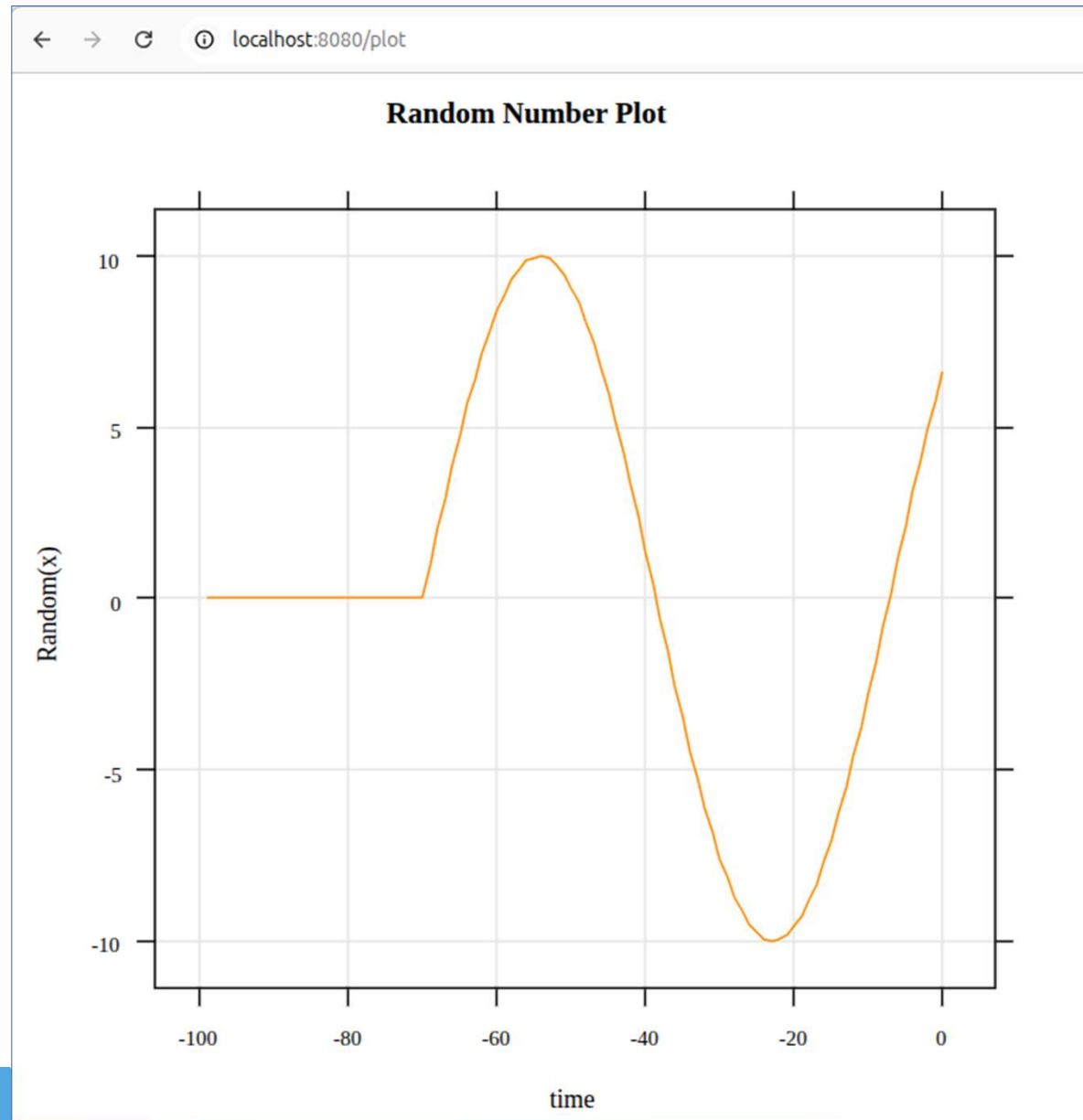
```
[INFO] --- maven-resources-plugin:3.1.0:resources (default-resources) @ spring-r ---
```

```
[INFO] Using 'UTF-8' encoding to copy filtered resources.
```

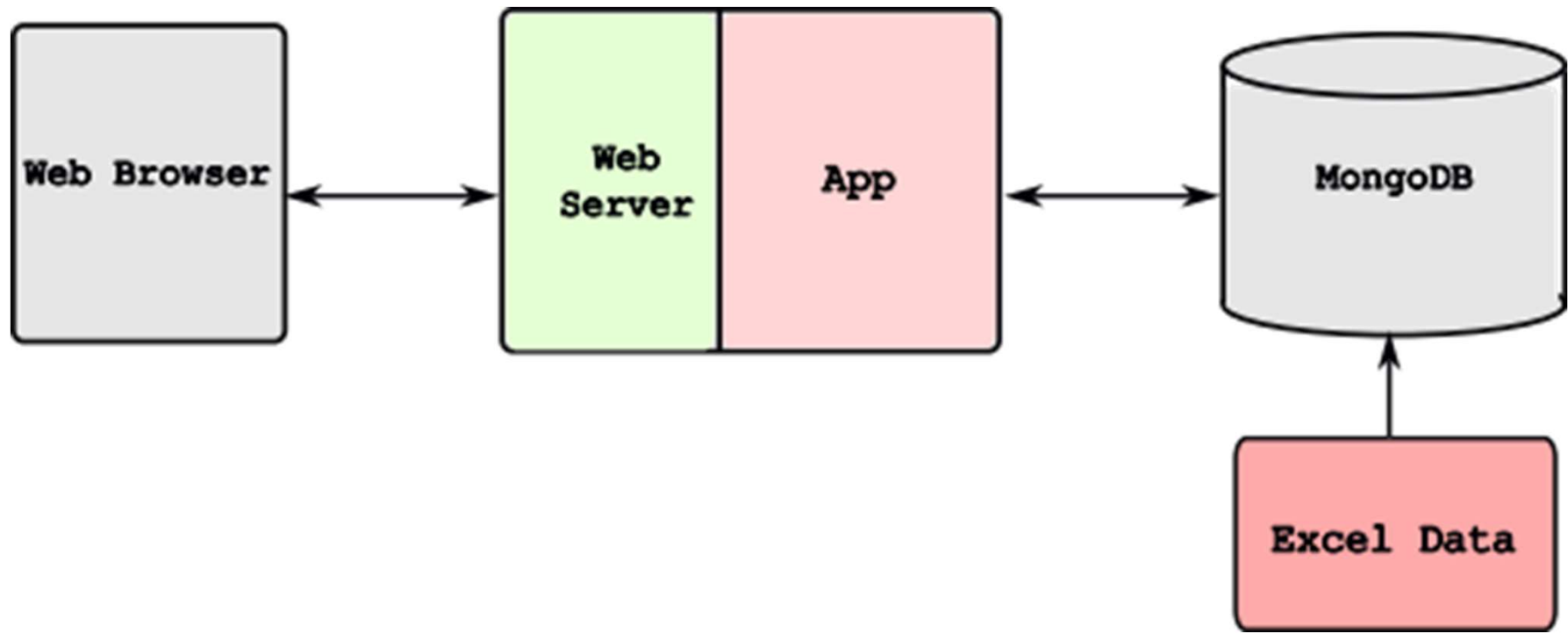
```
2025-10-01 22:47:58.048 INFO 10252 --- [main] o.s.web.context.ContextLoader : Root WebApplicationContext: initialization completed in 988 ms
function(dataHolder) {
  svg()
  data <- c(data[2:100], dataHolder$value)

  plot <- xyplot(randomData~time,
    data=data.frame(randomData = data, time = -99:0),
    main='Random Number Plot',
    ylab="Random(x)", type = c('l', 'g'), col.line='dark orange')
  print(plot)
  svg.off()
}
2025-10-01 22:48:14.631 INFO 10252 --- [main] o.s.s.concurrent.ThreadPoolTaskExecutor : Initializing ExecutorService 'applicationTaskExecutor'
2025-10-01 22:48:14.885 INFO 10252 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8080 (http) with context path ''
2025-10-01 22:48:14.889 INFO 10252 --- [main] o.g.demos.springr.SpringRApplication : Started SpringRApplication in 18.148 seconds (JVM running)
```

See output on the web



Project 1 Architecture



Questions?

